AGE 328: Communications and Computer Networking

Lecture 8: The Application Layer – Network Services, DNS, HTTP, and FTP

Dr. Kadeghe Fue, PhD, P.Eng (T)
Department of Agricultural Engineering
Sokoine University of Agriculture (SUA)
Morogoro, Tanzania

Abstract

The previous lectures navigated the complexities of moving bits across wires, routing packets across the globe, and establishing reliable end-to-end transport connections. However, none of that infrastructure holds value without the **Application Layer (Layer 7)**. This layer provides the interface between the network and the software applications that users interact with daily. This comprehensive compendium explores the core protocols that drive the modern internet and Agricultural Informatics: the Domain Name System (DNS), the Hypertext Transfer Protocol (HTTP), and the File Transfer Protocol (FTP). We will rigorously analyze network architectures (Client-Server vs. P2P), API payload structures for IoT sensors, and the mathematics of application performance and delay.

Contents

1 Introduction to the Application Layer **3**
1.1 Network Application Architectures . . . 3
1.1.1 1. Client-Server Architecture . . . 3
1.1.2 2. Peer-to-Peer (P2P) Architecture . . . 3
1.2 Services Demanded by Applications . . . 3

2 The Domain Name System (DNS): The Internet's Directory **5**
2.1 Why a Distributed Database? . . . 5
2.2 The Hierarchical Namespace . . . 5
2.3 DNS Caching . . . 6
2.4 DNS Record Types . . . 6

3 The World Wide Web and HTTP **7**
3.1 HTTP is Stateless . . . 7
3.2 HTTP Connections: Persistent vs. Non-Persistent . . . 7
3.2.1 Non-Persistent HTTP (HTTP/1.0) . . . 7
3.2.2 Persistent HTTP (HTTP/1.1 and beyond) . . . 7
3.3 HTTP Message Formats . . . 9
3.3.1 The HTTP Request Message . . . 9
3.3.2 The HTTP Response Message . . . 9
3.4 Case Study: RESTful APIs in Agriculture 4.0 . . . 10

4 File Transfer Protocol (FTP) 11
4.1 The Out-of-Band Control Architecture 11
4.2 Active vs. Passive FTP 11
4.2.1 Active FTP (The Legacy Method) 11
4.2.2 Passive FTP (The Modern Standard) 11
4.3 Agricultural Case Study: Drone Mapping Data Transfer 12

5 Electronic Mail (SMTP, POP3, IMAP) 13
5.1 The Three Components of Email 13
5.2 Simple Mail Transfer Protocol (SMTP) 13
5.3 Mail Access Protocols (POP3 vs. IMAP) 13
5.3.1 Post Office Protocol v3 (POP3) 13
5.3.2 Internet Message Access Protocol (IMAP) 14

6 Comprehensive Review and Engineering Exercises 15
6.1 Theoretical Questions 15
6.2 Numerical Laboratory Exercises 15

7 Glossary of Application Layer Terms 16

8 Academic References and Further Reading 16

1 Introduction to the Application Layer

The Application Layer is the topmost layer in both the OSI Reference Model (Layer 7) and the TCP/IP suite. It does not provide services to any other OSI layer; instead, it provides services directly to application processes running on the host operating system.

When an agricultural extension officer in Dodoma opens a web browser to check the latest maize prices on a government portal, the browser software itself is not the Application Layer. Rather, the *protocols* the browser uses to fetch that data (like HTTP and DNS) reside at the Application Layer.

1.1 Network Application Architectures

Before developing an application (such as a farm management dashboard), a software engineer must choose an architectural paradigm that dictates how the processes will communicate across the network.

1.1.1 1. Client-Server Architecture

This is the dominant architecture of the modern web.

- **Server:** An always-on host, with a permanent IP address. It waits passively for requests. It must be highly reliable and scalable. Example: The AWS cloud servers hosting the M-Kilimo database.
- **Client:** A host that initiates contact with the server. Clients may be intermittently connected and often have dynamic IP addresses. Clients do not communicate directly with other clients. Example: A farmer's smartphone or a tractor's onboard telemetry unit.

1.1.2 2. Peer-to-Peer (P2P) Architecture

There is no dedicated, always-on server. Instead, arbitrary end systems (peers) communicate directly with one another.

- **Advantages:** Highly scalable and self-organizing. As more peers join the network, they bring both demand and supply (bandwidth/storage).
- **Disadvantages:** Complex to manage, difficult to secure, and performance is highly variable based on peer availability.
- **Agricultural Application:** P2P architectures are increasingly used in **Blockchain** systems for agricultural supply chain tracing, where every farm, transport company, and processing plant acts as a peer maintaining a distributed ledger of coffee bean origins.

1.2 Services Demanded by Applications

When an Application Layer protocol passes data to the Transport Layer (TCP or UDP), what services does it require?

1. **Data Integrity:** Financial transactions (paying for fertilizer) require 100% reliable data transfer. (Requires TCP). Live audio streaming from a remote farm site can tolerate some loss. (Can use UDP).

2. **Timing / Latency:** Autonomous machinery control requires low-latency delivery (e.g., < 50 ms). Email can tolerate high latency.
3. **Throughput:** Streaming high-resolution drone video requires guaranteed minimum throughput. A soil moisture sensor only needs a few bytes per minute.
4. **Security:** Applications require confidentiality and data integrity, typically provided by encrypting the application payload using TLS (Transport Layer Security) before it hits TCP.

2 The Domain Name System (DNS): The Internet's Directory

While routers and switches use 32-bit IPv4 addresses (e.g., '196.216.71.10') or 128-bit IPv6 addresses, humans prefer alphanumeric names (e.g., 'www.sua.ac.tz'). The **Domain Name System (DNS)** is a distributed, hierarchical database that resolves human-readable host-names into IP addresses. It operates primarily over **UDP Port 53**.

2.1 Why a Distributed Database?

Why not have one massive server in New York containing a single text file mapping every website to its IP address?

- **Single Point of Failure:** If that server crashes, the global internet collapses.
- **Traffic Volume:** Handling billions of queries per second would overwhelm any single server network.
- **Distant Centralized Database:** A farmer in Morogoro trying to reach a server in Dar es Salaam would first have to send a query to New York, incurring massive latency.

2.2 The Hierarchical Namespace

DNS solves this by organizing servers in a strict hierarchy.

1. **Root DNS Servers:** At the top of the tree. There are 13 logical root server IP addresses (though physically there are hundreds of mirrored servers globally). They do not know the IP address of specific websites; they only know the IP addresses of the TLD servers.
2. **Top-Level Domain (TLD) Servers:** Responsible for domains like '.com', '.org', '.edu', and country-code domains (ccTLD) like '.tz' (Tanzania) or '.ke' (Kenya).
3. **Authoritative DNS Servers:** These are owned by organizations. SUA's authoritative DNS server holds the exact IP records for 'sua.ac.tz' and 'mail.sua.ac.tz'.

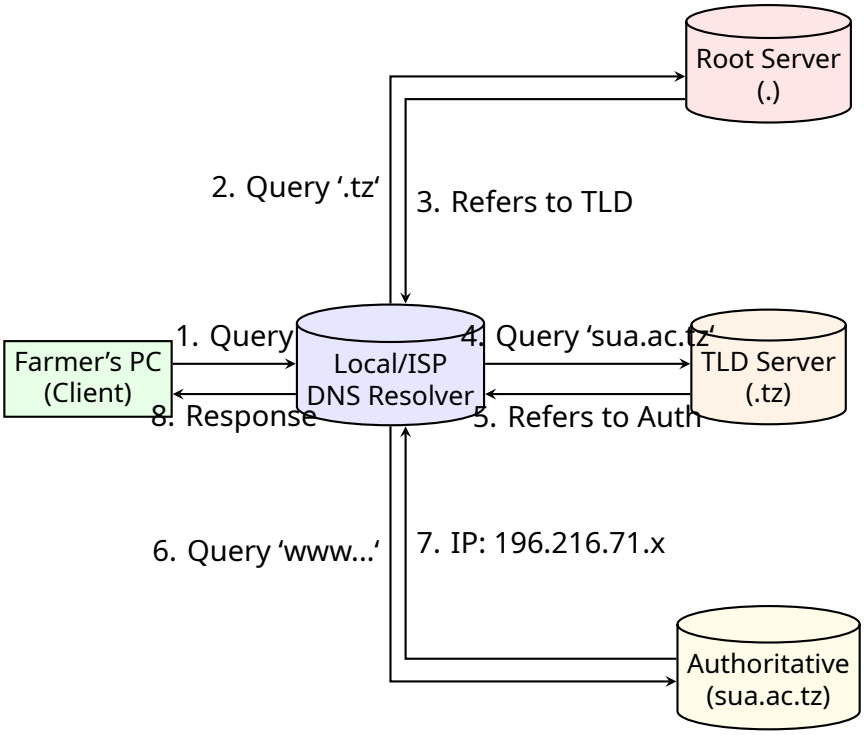


Figure 1: The Iterative DNS Resolution Process. The Local Resolver does the heavy lifting, querying the hierarchy step-by-step.

2.3 DNS Caching

To prevent bombarding the Root servers with every single request, DNS relies heavily on **caching**. When a Local DNS Resolver (e.g., operated by Vodacom Tanzania) learns the IP address of a TLD server or an Authoritative server, it caches that mapping in its memory for a defined **Time to Live (TTL)**. The next time a user asks for that domain, the Local Resolver answers immediately from its cache.

2.4 DNS Record Types

The authoritative servers hold various types of Resource Records (RR):

- **Type A:** Maps a hostname to an IPv4 address.
- **Type AAAA:** Maps a hostname to an IPv6 address.
- **Type NS:** Indicates the hostname of the authoritative Name Server for the domain.
- **Type MX:** Mail Exchange. Indicates the mail server responsible for accepting emails for the domain (e.g., routing 'user@sua.ac.tz' to the correct Google or Microsoft server).
- **Type CNAME:** Canonical Name. An alias mapping one hostname to another true hostname.

3 The World Wide Web and HTTP

The World Wide Web is an architectural framework for accessing linked documents. The protocol that powers the web is the **Hypertext Transfer Protocol (HTTP)**.

HTTP is implemented in two programs: a client program (a Web browser, or an IoT sensor's firmware) and a server program (like Apache or Nginx). They communicate by exchanging HTTP messages over **TCP Port 80** (or Port 443 for HTTPS).

3.1 HTTP is Stateless

A critical engineering feature of HTTP is that it is **Stateless**. The server maintains absolutely no information about past client requests. If a farmer requests the "Maize Prices" page, and then requests the exact same page 3 seconds later, the server processes the second request exactly as it did the first; it does not remember the farmer.

Why? Stateless protocols are highly scalable. If a server crashes and reboots, it doesn't lose thousands of "active session states" because none existed.

However, modern e-commerce (buying fertilizer online) requires state (e.g., a shopping cart). We solve this using **Cookies**—small text files stored on the client's machine that are sent back to the server with every HTTP request, allowing the server to identify the user.

3.2 HTTP Connections: Persistent vs. Non-Persistent

A web page usually consists of a base HTML file and several referenced objects (images, CSS scripts).

3.2.1 Non-Persistent HTTP (HTTP/1.0)

For every single object requested, a new TCP connection must be opened, the object transferred, and the TCP connection closed.

- **Penalty:** For a page with 1 HTML file and 10 images, 11 separate TCP 3-Way Handshakes must occur.
- **Delay Math:** Requesting a single object requires $2 \times RTT$ (Round Trip Times): One RTT to establish the TCP connection, and one RTT to send the HTTP request and receive the first bytes of the file, plus the actual transmission time of the file.

3.2.2 Persistent HTTP (HTTP/1.1 and beyond)

The server leaves the TCP connection open after sending a response. Subsequent requests and responses between the same client and server can be sent over that single connection.

- **Pipelining:** The client can blast multiple requests for all 10 images simultaneously over the open TCP connection without waiting for the first image to finish downloading.

Engineering Calculation: Web Page Delay Analysis

Scenario: An agricultural extension officer accesses a remote server to download a crop disease manual. The page consists of 1 base HTML file and 4 high-resolution reference images.

- RTT (Round Trip Time) = 150 ms.
- Transmission time of base HTML = 10 ms.
- Transmission time of each image = 50 ms.

Question 1: Calculate the total time to load the page using Non-Persistent HTTP (no parallel connections). For each object, time $= 2 \times RTT + T_{transmit}$. Base HTML: $(2 \times 150) + 10 = 310$ ms. For the 4 images: $4 \times [(2 \times 150) + 50] = 4 \times 350 = 1400$ ms. **Total Time** $= 310 + 1400 =$ **1710 ms**.

Question 2: Calculate using Persistent HTTP (with pipelining). 1st object (HTML) requires TCP setup: $2 \times RTT + T_{transmit} = 310$ ms. The TCP connection remains open. The browser requests all 4 images at once. The server sends them sequentially. Delay for all 4 images: $1 \times RTT + (4 \times 50) = 150 + 200 = 350$ ms. **Total Time** $= 310 + 350 =$ **660 ms**.

Conclusion: Persistent HTTP speeds up the page load by nearly $3\times$ on high-latency rural links.

3.3 HTTP Message Formats

HTTP messages are human-readable ASCII text. An agricultural software engineer must understand how to construct and parse these messages.

3.3.1 The HTTP Request Message

A request consists of a Request Line, Header Lines, an empty line, and an optional Entity Body.

```
GET /docs/maize_prices.pdf HTTP/1.1
Host: www.kilimo.go.tz
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept-Language: en-US,sw;q=0.9
Connection: keep-alive
```

HTTP Methods:

- **GET:** Requests a document from the server. (The most common method).
- **POST:** Submits data to be processed to a specified resource. (e.g., submitting an online form, or uploading sensor data).
- **PUT:** Replaces all current representations of the target resource with the uploaded content.
- **DELETE:** Deletes the specified resource.

3.3.2 The HTTP Response Message

The server replies with a Status Line, Header Lines, an empty line, and the actual Data.

```
HTTP/1.1 200 OK
Date: Tue, 04 Oct 2026 12:00:00 GMT
Server: Apache/2.4.41 (Ubuntu)
Last-Modified: Mon, 03 Oct 2026 08:30:00 GMT
Content-Length: 1048576
Content-Type: application/pdf

(raw binary data of the PDF file...)
```

HTTP Status Codes:

- **200 OK:** Request succeeded, and the requested object is in the message.
- **301 Moved Permanently:** The requested object has a new permanent URL.
- **400 Bad Request:** The server could not understand the request.
- **404 Not Found:** The requested document does not exist on this server.
- **500 Internal Server Error:** The server encountered an unexpected condition (e.g., the PHP or Python script crashed).

3.4 Case Study: RESTful APIs in Agriculture 4.0

HTTP is not just for humans viewing websites. It is the backbone of **Machine-to-Machine (M2M)** communication in IoT (Internet of Things).

Scenario: A smart greenhouse in Morogoro has a microcontroller managing soil moisture. Every 5 minutes, it sends a payload to a cloud server using a REST (REpresentational State Transfer) API via HTTP POST.

Instead of returning an HTML webpage, modern APIs use **JSON (JavaScript Object Notation)**, a lightweight data-interchange format.

The Sensor's HTTP Request:

```
POST /api/v1/sensors/upload HTTP/1.1
Host: cloud.smartfarm.tz
Content-Type: application/json
Content-Length: 85

{
  "sensor_id": "GH-01-MOIST",
  "timestamp": "2026-10-04T12:05:00Z",
  "moisture_level": 42.5
}
```

The server processes this JSON, stores it in an SQL database, and returns an 'HTTP 201 Created' status code. This seamless integration between embedded hardware and cloud web technologies is the foundation of modern agricultural engineering.

4 File Transfer Protocol (FTP)

While HTTP is optimized for quick, stateless transfers of small objects, the **File Transfer Protocol (FTP)**, defined in RFC 959, was designed specifically for the reliable transfer of massive files between a client and a remote host.

4.1 The Out-of-Band Control Architecture

The defining engineering characteristic of FTP is that it uses **two parallel TCP connections**:

1. **Control Connection (Port 21):** Used solely for sending commands (like 'USER name', 'PASS password', 'LIST', 'RETR filename') and receiving status replies. This connection remains open for the entire duration of the user's session.
2. **Data Connection (Port 20):** Used solely for transferring the actual file data. A new data connection is opened dynamically every time a file transfer is requested, and closed as soon as the file finishes transferring.

Because the control commands are sent on a separate TCP channel from the data, FTP is said to send its control information **out-of-band** (unlike HTTP, which sends headers and data over the exact same connection, known as in-band).

4.2 Active vs. Passive FTP

A classic networking problem arises with FTP when dealing with modern Firewalls and NAT (Network Address Translation) routers.

4.2.1 Active FTP (The Legacy Method)

- The Client opens a random dynamic port (e.g., Port 5000) and connects to the Server's Port 21 to establish the Control Connection.
- The Client sends a 'PORT 5001' command to the server, saying: "I am listening on Port 5001 for the data."
- The *Server* then initiates a TCP connection from its Port 20 to the Client's Port 5001.
- **The Firewall Problem:** Modern client firewalls block incoming connection attempts. When the server tries to connect to the client's Port 5001, the farmer's local router blocks the traffic, and the file transfer fails.

4.2.2 Passive FTP (The Modern Standard)

To solve the firewall issue, PASV mode was introduced.

- The Client connects to the Server's Port 21.
- The Client sends the 'PASV' command.
- The Server replies: "I am opening random Port 7050 for the data connection. Connect to me here."
- The *Client* then initiates the Data Connection to the Server's Port 7050.
- **Why it works:** Firewalls generally allow outgoing connections. Because the client initiates both the control and data connections, the firewall permits the traffic.

4.3 Agricultural Case Study: Drone Mapping Data Transfer

Scenario: An agricultural surveying company flies a fixed-wing drone over the vast sugarcane fields of the Kilombero Valley. The drone captures multispectral imagery, generating a 50 GB GeoTIFF orthomosaic file.

Why HTTP fails here: The engineer needs to upload this 50 GB file to the analytics server in Dar es Salaam over a potentially unstable rural WiMAX link. If they use HTTP (e.g., a web browser upload form), a single dropped connection might force the 50 GB upload to restart from zero. Web browsers are terrible at managing massive file states.

The FTP Solution: The engineer uses an FTP client. FTP inherently supports **state**. The server remembers the current directory and the user's authentication status. More importantly, FTP clients support **Resume capability**. If the WiMAX link drops at 45 GB, the client reconnects and issues the 'REST' (Restart) command, telling the server to append new data starting exactly at byte 45,000,000,001.

Note on Security: Standard FTP transmits passwords in plaintext. In modern engineering, we universally replace it with **SFTP (SSH File Transfer Protocol)**, which encrypts the entire session using secure shell cryptography, preventing packet sniffers from stealing server credentials.

5 Electronic Mail (SMTP, POP3, IMAP)

Email remains the most critical asynchronous communication tool in business and administration. Unlike HTTP, which is a ”pull” protocol (the user pulls data from the server), email relies heavily on a ”push” protocol.

5.1 The Three Components of Email

1. **User Agents (MUA):** The software the user interacts with (e.g., Microsoft Outlook, Mozilla Thunderbird, or a webmail interface).

2. **Mail Servers (MTA/MDA):** The core infrastructure. They contain the mailboxes for users and maintain message queues for outgoing mail.

3. **Protocols:** SMTP, POP3, and IMAP.

5.2 Simple Mail Transfer Protocol (SMTP)

SMTP is the principal application-layer protocol for Internet electronic mail. It uses the reliable data transfer service of **TCP on Port 25**.

SMTP is strictly a **Push protocol**. It is used in two specific scenarios:

- Transferring mail from the sender’s User Agent to the sender’s Mail Server.

- Transferring mail from the sender’s Mail Server to the recipient’s Mail Server.

> **SMTP Architecture Detail:** SMTP requires messages to be entirely in 7-bit ASCII format. If a user attaches a high-resolution PDF map of a farm layout, that binary file must be encoded into ASCII text using **MIME (Multipurpose Internet Mail Extensions)** (specifically Base64 encoding) before SMTP can push it to the server.

5.3 Mail Access Protocols (POP3 vs. IMAP)

Once SMTP delivers the email to the recipient’s Mail Server, how does the user read it? SMTP cannot be used by the recipient to ”pull” messages down to their laptop.

5.3.1 Post Office Protocol v3 (POP3)

A very simple protocol operating on **TCP Port 110**. It essentially operates in a ”download and delete” mode.

- The user agent connects to the server, downloads all new messages to the local hard drive, and then tells the server to delete them from the server.

- **Problem:** If an extension officer reads an email on their office PC, it is deleted from the server. When they check their phone in the field later, the email is not there.

5.3.2 Internet Message Access Protocol (IMAP)

A much more complex protocol operating on **TCP Port 143**.

- IMAP leaves the messages on the server. The user agent simply syncs a view of the server's state.
- Users can create folders, move messages, and search the server's database remotely.
- State is maintained across multiple devices (PC, smartphone, tablet). If an email is read on the phone, it is marked as ”Read” on the server, and the PC will also show it as ”Read.”

6 Comprehensive Review and Engineering Exercises

6.1 Theoretical Questions

1. Compare and contrast the Client-Server architecture with the Peer-to-Peer (P2P) architecture. Discuss a scenario in agricultural data management where P2P would be advantageous.
2. Explain the difference between an Iterative and a Recursive DNS query. Why do Root DNS servers typically refuse to perform recursive queries for local resolvers?
3. HTTP is fundamentally a ”stateless” protocol. Explain what this means. How do web applications manage to maintain a ”session” (like a logged-in user profile) on top of a stateless protocol?
4. Describe the out-of-band control architecture of FTP. Why does Active FTP fail when the client is behind a standard NAT router or firewall, and how does Passive FTP solve this issue?
5. Why are three distinct protocols (SMTP, POP3, IMAP) required for a complete email ecosystem? Why can't SMTP handle the entire process from sender to receiver?

6.2 Numerical Laboratory Exercises

1. **HTTP Delay Calculation:** An agricultural engineer in Mbeya requests a webpage hosted on a server in Europe. The webpage consists of a base HTML file and 5 embedded images.
   - The Round Trip Time (RTT) between Mbeya and the European server is 200 ms.
   - The transmission time for the base HTML file is 20 ms.
   - The transmission time for *each* image is 50 ms.
   - DNS lookup time (total delay to resolve the hostname) is 100 ms.

   Task A: Calculate the total time elapsed from when the user clicks the link to when the entire page is fully loaded, assuming the browser uses **Non-Persistent HTTP/1.0** with no parallel connections.

   Task B: Calculate the total time elapsed assuming the browser uses **Persistent HTTP/1.1** with pipelining (all images requested simultaneously on one open TCP connection).

   Task C: What percentage of time was saved by upgrading to Persistent HTTP?
2. **DNS Caching Math:** A Local DNS Resolver receives queries for 'www.sua.ac.tz' at a rate of $\lambda = 10$ queries per minute. The Authoritative server for SUA sets the Time To Live (TTL) for this record to 3600 seconds (1 hour).
   (a) If the local cache is currently empty, how many times will the Local Resolver have to query the remote Authoritative server over a continuous 24-hour period?
   (b) What percentage of user queries will be served directly from the cache (cache hit rate) during that 24 hours?

7 Glossary of Application Layer Terms

Application Programming Interface (API): A set of rules and protocols that allows different software applications to communicate with each other. REST is a common architectural style for web APIs.

Authoritative DNS Server: A name server that holds the actual DNS records (A, MX, CNAME) for a specific domain, rather than a cached copy.

Base64 Encoding: A mechanism for converting binary data (like images or PDF attachments) into a string of ASCII characters, allowing it to be transmitted over text-based protocols like SMTP.

JSON (JavaScript Object Notation): A lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate. Heavily used in HTTP REST APIs.

Stateless Protocol: A communications protocol in which no session information is retained by the receiver. Each request is treated as an independent transaction.

8 Academic References and Further Reading

1. **Kurose, J. F., & Ross, K. W. (2021).** *Computer Networking: A Top-Down Approach* (8th ed.). Pearson. (Chapter 2 is entirely dedicated to the Application Layer, providing exceptional detail on HTTP, DNS, and Socket Programming).
2. **Fielding, R. T. (2000).** *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, UC Irvine. (The foundational text defining RESTful API architecture).
3. **RFC 2616:** Hypertext Transfer Protocol – HTTP/1.1. (The technical specification governing web traffic).
4. **RFC 1034 & RFC 1035:** Domain Names - Concepts and Facilities. (The foundational documents for DNS).

End of Lecture 8 Compendium.
Prepared for the Department of Agricultural Engineering, SUA.